

SIMATS ENGINEERING

Department of Computer Science & Engineering

ASSESSMENT TOOL 1- EXPERIMENTS

Course Code:	CSA1209 & CSA1215	Course Title:	Computer Architecture
CO Assessed:	CO3	Assessment:	ASSESSMENT TOOL 1- EXPERIMENTS
Weightage:	40%	Total Marks:	25
CO3 Statement:	<i>Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT). (BL6)</i>		
Student Name:	D. Gnana Deepthi	Reg.No.:	192411123
Department:	CSE	Date:	

ANNEXURE A

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyze how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.

4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
 5. Analyze a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.
-

Code of Conduct:

I D. Gnana Deepthi (192411123) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: D. Gnanadeepthi

ANSWERS

1.

Code:

Register Transfer and ALU Operations Simulation

Initial register values

R1 = 22

R2 = 9

R3 = 0

R4 = 0

```
print("Initial Register Values:")
print("R1 =", R1)
print("R2 =", R2)
print("R3 =", R3)
print("R4 =", R4)

# 1. Register Transfer
R3 = R1
print("\n1. Register Transfer: R3 <- R1")
print("R3 =", R3)

# 2. Addition
R4 = R1 + R2
print("\n2. Addition: R4 <- R1 + R2")
print("R4 =", R4)

# 3. Subtraction
R4 = R1 - R2
print("\n3. Subtraction: R4 <- R1 - R2")
print("R4 =", R4)

# 4. AND Operation
R4 = R1 & R2
print("\n4. AND Operation: R4 <- R1 AND R2")
print("R4 =", R4)

# 5. OR Operation
R4 = R1 | R2
print("\n5. OR Operation: R4 <- R1 OR R2")
print("R4 =", R4)
```

OUTPUT:

```
Output

Initial Register Values:
R1 = 22
R2 = 9
R3 = 0
R4 = 0

1. Register Transfer: R3 <- R1
R3 = 22

2. Addition: R4 <- R1 + R2
R4 = 31

3. Subtraction: R4 <- R1 - R2
R4 = 13

4. AND Operation: R4 <- R1 AND R2
R4 = 0

5. OR Operation: R4 <- R1 OR R2
R4 = 31

=== Code Execution Successful ===
```

2.

CODE:

```
# Single-Bus and Multi-Bus Computer System Simulation
```

```
# ----- SINGLE BUS -----
```

```
print("----- SINGLE BUS ORGANIZATION -----")
```

```
cycles_single = 0
```

```
R1 = 10
```

```
R2 = 20
```

```
R3 = 0
```

```
memory = {100: 0}
```

```
io_device = 0
```

```
# Register to Register Transfer
```

```
R3 = R1
```

```
cycles_single += 1
```

```
print("R3 <- R1 :", R3)
```

```
# Memory Transfer
```

```
memory[100] = R2
```

```
cycles_single += 1
```

```
print("Memory[100] <- R2 :", memory[100])
```

```
# Memory to Register Transfer
```

```
R1 = memory[100]
```

```
cycles_single += 1
```

```
print("R1 <- Memory[100] :", R1)
```

```
# Register to I/O Transfer
```

```
io_device = R3
```

```
cycles_single += 1
```

```
print("I/O Device <- R3 :", io_device)
```

```
print("Total clock cycles for Single Bus =", cycles_single)
```

```
# ----- MULTI BUS -----
```

```
print("\n----- MULTI BUS ORGANIZATION -----")
```

```
cycles_multi = 0
```

```
R1 = 10
```

```
R2 = 20
```

```
R3 = 0
```

```
memory = {100: 0}
```

```
io_device = 0
```

```
# Parallel transfers using multiple buses
```

```
# Bus 1: R3 <- R1
```

```
# Bus 2: Memory[100] <- R2
```

```
R3 = R1
```

```
memory[100] = R2
```

```
cycles_multi += 1
```

```
print("R3 <- R1 :", R3)
```

```
print("Memory[100] <- R2 :", memory[100])
```

```
# Bus 1: R1 <- Memory[100]
```

```
# Bus 2: I/O Device <- R3
```

```
R1 = memory[100]
```

```
io_device = R3
```

```
cycles_multi += 1
```

```
print("R1 <- Memory[100] :", R1)
```

```
print("I/O Device <- R3 :", io_device)
```

```
print("Total clock cycles for Multi Bus =", cycles_multi)
```

```
# ----- COMPARISON -----
```

```
print("\n----- COMPARISON -----")
```

```
print("Single Bus Time (cycles):", cycles_single)
```

```
print("Multi Bus Time (cycles):", cycles_multi)
```

```
if cycles_multi < cycles_single:
```

```
    print("Multi-bus organization is faster and more efficient.")
```

```
else:
```

```
    print("Single-bus organization is equally or more efficient.")
```

```
print("\nEfficiency Improvement:")
```

```
improvement = ((cycles_single - cycles_multi) / cycles_single) * 100
```

```
print("Multi-bus reduces execution time by", improvement, "%")
```

OUTPUT:

Output

```
R3 <- R1 : 10
Memory[100] <- R2 : 20
R1 <- Memory[100] : 20
I/O Device <- R3 : 10
Total clock cycles for Single Bus = 4

----- MULTI BUS ORGANIZATION -----
R3 <- R1 : 10
Memory[100] <- R2 : 20
R1 <- Memory[100] : 20
I/O Device <- R3 : 10
Total clock cycles for Multi Bus = 2

----- COMPARISON -----
Single Bus Time (cycles): 4
Multi Bus Time (cycles): 2
Multi-bus organization is faster and more efficient.

Efficiency Improvement:
Multi-bus reduces execution time by 50.0 %

=== Code Execution Successful ===
```

3.

CODE:

5-Stage Instruction Pipeline Simulation

```
instructions = [
    "ADD R1, R2, R3",
    "SUB R4, R1, R5",
    "AND R6, R2, R3",
```



```

    "OR R7, R4, R1"
]

stages = ["IF", "ID", "EX", "MEM", "WB"]

n = len(instructions)
pipeline_cycles = n + len(stages) - 1
non_pipeline_cycles = n * len(stages)

print("----- 5-STAGE PIPELINE SIMULATION -----\\n")

# Display pipeline execution table
print("Cycle\\t", end="")

for cycle in range(1, pipeline_cycles + 1):
    print("C" + str(cycle) + "\\t", end="")

print()

for i in range(n):
    print("I" + str(i + 1) + "\\t", end="")

    for cycle in range(1, pipeline_cycles + 1):
        stage_index = cycle - i - 1

        if 0 <= stage_index < len(stages):
            print(stages[stage_index] + "\\t", end="")
        else:
            print("-\\t", end="")

```

print()

Performance calculation

speedup = non_pipeline_cycles / pipeline_cycles

throughput_pipeline = n / pipeline_cycles

throughput_non_pipeline = n / non_pipeline_cycles

print("\n----- PERFORMANCE ANALYSIS -----")

print("Number of Instructions =", n)

print("Pipeline Stages =", len(stages))

print("\nPipelined System:")

print("Total Clock Cycles =", pipeline_cycles)

print("Throughput =", round(throughput_pipeline, 2), "instructions/cycle")

print("\Non-Pipelined System:")

print("Total Clock Cycles =", non_pipeline_cycles)

print("Throughput =", round(throughput_non_pipeline, 2), "instructions/cycle")

print("\nSpeedup =", round(speedup, 2), "times")

OUTPUT:

```
Output

Cycle   C1   C2   C3   C4   C5   C6   C7   C8
I1  IF   ID   EX  MEM  WB   -   -   -
I2   -   IF   ID   EX  MEM  WB   -   -
I3   -   -   IF   ID   EX  MEM  WB   -
I4   -   -   -   IF   ID   EX  MEM  WB

----- PERFORMANCE ANALYSIS -----
Number of Instructions = 4
Pipeline Stages = 5

Pipelined System:
Total Clock Cycles = 8
Throughput = 0.5 instructions/cycle

Non-Pipelined System:
Total Clock Cycles = 20
Throughput = 0.2 instructions/cycle

Speedup = 2.5 times

=== Code Execution Successful ===
```

4,

CODE:

Pipeline Hazard Simulation

```
instructions = [
    "I1: LOAD R1, 100(R2)",
    "I2: ADD R3, R1, R4",    # Data hazard with I1
    "I3: STORE R3, 200(R5)", # Structural/Data hazard
    "I4: BEQ R3, R0, TARGET", # Control hazard
    "I5: SUB R6, R3, R7"
]
```

```

print("----- INSTRUCTION SET -----")

for instruction in instructions:

    print(instruction)


# Normal 5-stage pipeline

normal_cycles = len(instructions) + 4


# Hazards without techniques

data_stall = 2      # Data hazard causes stalling

structural_stall = 1 # Structural hazard causes stalling

branch_penalty = 2  # Control hazard penalty


cycles_without_solution = (

    normal_cycles +

    data_stall +

    structural_stall +

    branch_penalty

)


print("\n----- HAZARDS DETECTED -----")

print("Data Hazard: I2 depends on R1 produced by I1")

print("Structural Hazard: Memory access conflict between instructions")

print("Control Hazard: Branch instruction I4 may change execution path")


print("\n----- WITHOUT HAZARD RESOLUTION -----")

print("Base Pipeline Cycles =", normal_cycles)

print("Data Hazard Stall =", data_stall)

print("Structural Hazard Stall =", structural_stall)

print("Control Hazard Penalty =", branch_penalty)

```

```

print("Total Cycles =", cycles_without_solution)

# Hazard resolution techniques

# Data forwarding reduces data stall
data_stall_with_forwarding = 0

# Separate instruction and data memory reduces structural hazard
structural_stall_resolved = 0

# Branch prediction reduces branch penalty
branch_penalty_with_prediction = 1

cycles_with_solution = (
    normal_cycles +
    data_stall_with_forwarding +
    structural_stall_resolved +
    branch_penalty_with_prediction
)

print("\n----- HAZARD RESOLUTION TECHNIQUES -----")
print("Data Forwarding: Used to pass result directly to next instruction")
print("Structural Hazard Solution: Separate memory/bus resources used")
print("Branch Prediction: Predicts branch direction before execution")

print("\n----- WITH HAZARD RESOLUTION -----")
print("Base Pipeline Cycles =", normal_cycles)
print("Data Stall after Forwarding =", data_stall_with_forwarding)
print("Structural Stall after Solution =", structural_stall_resolved)

```

```
print("Branch Penalty after Prediction =", branch_penalty_with_prediction)
print("Total Cycles =", cycles_with_solution)
```

```
# Performance comparison
```

```
speedup = cycles_without_solution / cycles_with_solution
```

```
improvement = (
    (cycles_without_solution - cycles_with_solution)
    / cycles_without_solution
) * 100
```

```
print("\n----- PERFORMANCE COMPARISON -----")
```

```
print("Cycles Without Resolution =", cycles_without_solution)
```

```
print("Cycles With Resolution =", cycles_with_solution)
```

```
print("Speedup =", round(speedup, 2), "times")
```

```
print("Performance Improvement =", round(improvement, 2), "%")
```

OUTPUT:

```
Output

Control Hazard Penalty = 2
Total Cycles = 14

----- HAZARD RESOLUTION TECHNIQUES -----
Data Forwarding: Used to pass result directly to next instruction
Structural Hazard Solution: Separate memory/bus resources used
Branch Prediction: Predicts branch direction before execution

----- WITH HAZARD RESOLUTION -----
Base Pipeline Cycles = 9
Data Stall after Forwarding = 0
Structural Stall after Solution = 0
Branch Penalty after Prediction = 1
Total Cycles = 10

----- PERFORMANCE COMPARISON -----
Cycles Without Resolution = 14
Cycles With Resolution = 10
Speedup = 1.4 times
Performance Improvement = 28.57 %

=== Code Execution Successful ===
```

5.

CODE:

Superscalar, Out-of-Order, Speculative Execution and SMT Simulation

instructions = 12

issue_width = 2 # Superscalar processor can execute 2 instructions/cycle

threads = 2 # Hyper-threading / SMT uses 2 threads

print("----- PROCESSOR EXECUTION ANALYSIS -----\\n")

1. Normal Scalar Processor

scalar_cycles = instructions

scalar_throughput = instructions / scalar_cycles

scalar_utilization = 50

print("1. SCALAR PROCESSOR")

print("Instructions =", instructions)

print("Cycles =", scalar_cycles)

print("Throughput =", round(scalar_throughput, 2), "instructions/cycle")

print("Resource Utilization =", scalar_utilization, "%\n")

2. Superscalar Processor

superscalar_cycles = (instructions + issue_width - 1) // issue_width

superscalar_throughput = instructions / superscalar_cycles

superscalar_utilization = 80

print("2. SUPERSCALAR PROCESSOR")

print("Issue Width =", issue_width)

print("Cycles =", superscalar_cycles)

print("Throughput =", round(superscalar_throughput, 2), "instructions/cycle")

print("Resource Utilization =", superscalar_utilization, "%\n")

3. Out-of-Order Execution

Independent instructions execute while dependent instructions wait

ooo_cycles = superscalar_cycles - 1


```
ooo_throughput = instructions / ooo_cycles
```

```
ooo_utilization = 85
```

```
print("3. OUT-OF-ORDER EXECUTION")
```

```
print("Cycles =", ooo_cycles)
```

```
print("Throughput =", round(ooo_throughput, 2), "instructions/cycle")
```

```
print("Resource Utilization =", ooo_utilization, "%\n")
```

```
# 4. Speculative Execution
```

```
# Correct branch prediction reduces waiting time
```

```
speculative_cycles = ooo_cycles - 1
```

```
speculative_throughput = instructions / speculative_cycles
```

```
speculative_utilization = 88
```

```
print("4. SPECULATIVE EXECUTION")
```

```
print("Cycles =", speculative_cycles)
```

```
print("Throughput =", round(speculative_throughput, 2), "instructions/cycle")
```

```
print("Resource Utilization =", speculative_utilization, "%\n")
```

```
# 5. Hyper-Threading
```

```
# Two threads share processor resources
```

```
ht_cycles = speculative_cycles
```

```
ht_instructions = instructions * threads
```

```
ht_throughput = ht_instructions / ht_cycles
```

```
ht_utilization = 92
```

```
print("5. HYPER-THREADING")
print("Threads =", threads)
print("Instructions Executed =", ht_instructions)
print("Cycles =", ht_cycles)
print("Throughput =", round(ht_throughput, 2), "instructions/cycle")
print("Resource Utilization =", ht_utilization, "%\n")
```

6. Simultaneous Multithreading (SMT)

Instructions from both threads can issue in the same cycle

```
smt_cycles = 4
smt_instructions = instructions * threads
smt_throughput = smt_instructions / smt_cycles
smt_utilization = 96
```

```
print("6. SIMULTANEOUS MULTITHREADING (SMT)")
print("Threads =", threads)
print("Instructions Executed =", smt_instructions)
print("Cycles =", smt_cycles)
print("Throughput =", round(smt_throughput, 2), "instructions/cycle")
print("Resource Utilization =", smt_utilization, "%\n")
```

Performance Comparison

```
print("----- PERFORMANCE COMPARISON -----")
print("Scalar Throughput    =", round(scalar_throughput, 2))
```

```

print("Superscalar Throughput =", round(superscalar_throughput, 2))
print("OoO Throughput      =", round(ooo_throughput, 2))
print("Speculative Throughput =", round(speculative_throughput, 2))
print("Hyper-Threading     =", round(ht_throughput, 2))
print("SMT Throughput       =", round(smt_throughput, 2))

```

```

speedup = smt_throughput / scalar_throughput

```

```

print("\nOverall SMT Speedup =", round(speedup, 2), "times")

```

OUTPUT:

```

Output

Cycles = 4
Throughput = 6.0 instructions/cycle
Resource Utilization = 92 %

6. SIMULTANEOUS MULTITHREADING (SMT)
Threads = 2
Instructions Executed = 24
Cycles = 4
Throughput = 6.0 instructions/cycle
Resource Utilization = 96 %

----- PERFORMANCE COMPARISON -----
Scalar Throughput      = 1.0
Superscalar Throughput = 2.0
OoO Throughput         = 2.4
Speculative Throughput = 3.0
Hyper-Threading        = 6.0
SMT Throughput         = 6.0

Overall SMT Speedup = 6.0 times

=== Code Execution Successful ===

```

MARKS ALLOCATION

	Understanding of Concepts (20%)	Experimental Design (20%)	Use of Tools (25%)	Analysis of Results (20%)	Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate	Good analysis with	Basic analysis	Poor or incorrect

		interpretation and validation of results	reasonable interpretation	with limited insights	analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation